UNIVERSIDADE DO VALE DO RIO DOS SINOS (UNISINOS)

CURSO DE ENGENHARIA DA COMPUTAÇÃO

GUIA DE DOMÍNIO TÉCNICO E DEFESA DE BANCA

MÓDULO 3: Algoritmo Genético Aprimorado (IGA)

Otimização Conjunta $\longrightarrow$ Cromossomo Híbrido $\longrightarrow$ Crossover BLX-α $\longrightarrow$ Validação Cruzada Estratificada

---

Candidato: Andrio Gabriel Epping | **Papel:** Orientador Técnico & Examinador Rigoroso de Banca |
Data: Novembro de 2026

Apresentação do Módulo de Treinamento

Este guia constitui o material de domínio aprofundado do **Módulo 3: Algoritmo Genético Aprimorado (Improved Genetic Algorithm – IGA)**, o mecanismo central de inteligência computacional responsável pela seleção de variáveis e ajuste simultâneo de hiperparâmetros do classificador XGBoost nos eixos Track A e Track B do TCC.

O objetivo deste módulo é capacitar o estudante a explicar e defender, com rigor matemático e rastreabilidade linha a linha no código-fonte, a modelagem formal do cromossomo híbrido (binário e contínuo), as equações dos operadores genéticos (Torneio, Crossover Uniforme, Crossover BLX-α, Mutação Bit-Flip e Gaussiana), a garantia de preservação por elitismo, a integração com o `ImbPipeline` para evitar vazamento de dados via `CustomSMOTE`, e as respostas exatas para as arguições teóricas da banca examinadora.

Sumário

1 Pilar 1: Fundamentação Matemática e Teórica **3**
1.1 A Necessidade da Otimização Conjunta (Co-Otimização) . . . 3
1.2 Representação Matemática do Cromossomo Híbrido . . . 3
1.3 Inicialização Enviesada por Conhecimento Físico de Domínio . . . 4
1.4 Operadores Genéticos do IGA . . . 4
1.5 Função de Aptidão (Fitness) e Validação Cruzada Estratificada . . . 5

2 Pilar 2: Rastreamento no Código (Linha a Linha) **5**
2.1 Mapeamento de Arquivos e Estruturas . . . 5
2.2 Rastreamento da Decodificação Afim (`src/iga_xgboost/cromossomo.py`) . . . 6
2.3 Rastreamento dos Operadores Genéticos (`src/iga_xgboost/iga.py`) . . . 6
2.4 Rastreamento do Pipeline de Avaliação de Fitness (`src/iga_xgboost/iga.py`) . . . 7
2.5 Tabela de Dimensões e Configurações por Eixo . . . 7

3 Pilar 3: Decisões de Projeto e Nuances do Meu Trabalho **8**
3.1 Por que IGA e não Otimização Bayesiana (Optuna) ou GridSearchCV? . . . 8
3.2 O Papel Físico da Inicialização Enviesada ($p_{\text{iec}} = 0{,}80$) . . . 8
3.3 A Relação entre Complexidade Computacional e Ganho Diagnóstico . . . 8
3.4 Alinhamento Rigoroso entre Código e Texto do TCC . . . 9

4 Pilar 4: Sabatina da Banca (3 Perguntas Difíceis e Gabaritos) 9

1 Pilar 1: Fundamentação Matemática e Teórica

1.1 A Necessidade da Otimização Conjunta (Co-Otimização)

No treinamento de modelos complexos de aprendizado de máquina supervisionado para matrizes de Análise de Gases Dissolvidos (DGA), a seleção de variáveis e o ajuste de hiperparâmetros são problemas **fortemente acoplados e não-lineares**:

- A relevância de uma dada razão de gases (ex: CH_4/H_2 ou C_2H_2/C_2H_4) depende diretamente da profundidade máxima permitida para as árvores (`max_depth`) e da taxa de subamostragem por coluna (`colsample_bytree`).
- Separar o processo em duas etapas isoladas (ex: executar um algoritmo de filtro como SelectKBest ou RFE e, posteriormente, calibrar o XGBoost via busca em grade ou busca aleatória) induz sub-otimalidade severa, pois os hiperparâmetros serão calibrados para um conjunto fixo de atributos que pode não ser ideal para a topologia das árvores.

O IGA aborda essa dependência modelando a seleção de variáveis e os hiperparâmetros como um problema de otimização combinatória mista:

$$\mathbf{c}^* = \arg\max_{\mathbf{c}\in\Omega} \mathcal{F}(\mathbf{c}) \tag{1}$$

onde $\Omega = \{0,1\}^D \times [0,1]^K$ representa o espaço de busca conjunto.

1.2 Representação Matemática do Cromossomo Híbrido

Cada indivíduo da população é formalizado por um vetor unidimensional de números reais de precisão dupla (`float64`):

$$\mathbf{c} = \begin{bmatrix} \mathbf{m} \\ \boldsymbol{\gamma} \end{bmatrix} = \Big[m_1, m_2, \ldots, m_D,\ \gamma_1, \gamma_2, \ldots, \gamma_K \Big]^\top \in \mathbb{R}^{D+K} \tag{2}$$

onde:

- $\mathbf{m} \in \{0.0, 1.0\}^D$ é a máscara booleana de atributos:

$$m_j = \begin{cases} 1.0, & \text{se a feature } j \text{ está ativa (selecionada)} \\ 0.0, & \text{se a feature } j \text{ está inativa (descartada)} \end{cases} \tag{3}$$

- $\boldsymbol{\gamma} \in [0.0, 1.0]^K$ $(K = 9)$ é o vetor de genes contínuos normalizados correspondentes aos hiperparâmetros do XGBoost.

Espaço de Busca dos Hiperparâmetros do XGBoost ($K = 9$)

Cada gene contínuo $\gamma_k \in [0.0, 1.0]$ é mapeado linearmente para a escala física do algoritmo pela transformação afim:

$$\theta_k = g_k(\gamma_k) = \begin{cases} \text{round}\left(\theta_k^{\min} + \gamma_k \cdot (\theta_k^{\max} - \theta_k^{\min})\right), & \text{se tipo}(\theta_k) = \text{inteiro} \\ \theta_k^{\min} + \gamma_k \cdot (\theta_k^{\max} - \theta_k^{\min}), & \text{se tipo}(\theta_k) = \text{real} \end{cases} \tag{4}$$

Os limites estabelecidos no projeto (`src/iga_xgboost/cromossomo.py`) são:

1. $\theta_1 = \eta$ (taxa de aprendizado / `learning_rate`) : $[0{,}01;\ 0{,}30] \subset \mathbb{R}$
2. $\theta_2 = d_{\max}$ (profundidade máxima / `max_depth`) : $[3;\ 12] \subset \mathbb{Z}$
3. $\theta_3 = M$ (número de estimadores / `n_estimators`) : $[50;\ 500] \subset \mathbb{Z}$

4. $\theta_4 = s_{\text{row}}$ (subamostragem de linhas / `subsample`) : $[0{,}50;\ 1{,}00] \subset \mathbb{R}$
5. $\theta_5 = s_{\text{col}}$ (subamostragem de colunas / `colsample_bytree`) : $[0{,}30;\ 1{,}00] \subset \mathbb{R}$
6. $\theta_6 = \gamma_{\text{split}}$ (ganho mínimo de partição / `gamma`) : $[0{,}0;\ 5{,}0] \subset \mathbb{R}$
7. $\theta_7 = w_{\text{child}}$ (peso mínimo da folha / `min_child_weight`) : $[1;\ 10] \subset \mathbb{Z}$
8. $\theta_8 = \alpha_{L_1}$ (termo de regularização Lasso / `reg_alpha`) : $[0{,}0;\ 1{,}0] \subset \mathbb{R}$
9. $\theta_9 = \lambda_{L_2}$ (termo de regularização Ridge / `reg_lambda`) : $[0{,}0;\ 5{,}0] \subset \mathbb{R}$

1.3 Inicialização Enviesada por Conhecimento Físico de Domínio

Ao contrário dos AGs clássicos que sorteiam bits com probabilidade uniforme $p = 0{,}50$, o IGA injeta conhecimento físico prévio da química de isolamento na geração da população inicial ($g = 0$):

$$P(m_j = 1) = \begin{cases} 0{,}80, & \text{se a feature } j \text{ for uma razão consagrada (IEC, Rogers, Duval)} \\ 0{,}50, & \text{para os demais gases brutos ou somas parciais} \end{cases} \tag{5}$$

Essa inicialização garante que a busca inicie explorando bacias de atração termodinamicamente consistentes, acelerando a convergência e evitando que indivíduos com zero razões físico-químicas dominem as gerações iniciais.

1.4 Operadores Genéticos do IGA

Formulação dos Operadores Genéticos

1. Seleção por Torneio ($k = 3$):
Para selecionar cada progenitor, sorteiam-se 3 candidatos com reposição uniforme $\{i_1, i_2, i_3\} \sim \mathcal{U}\{1, P\}$ da população atual. O indivíduo selecionado é o de maior aptidão:

$$i^* = \arg\max_{i \in \{i_1, i_2, i_3\}} \mathcal{F}(\mathbf{c}_i) \tag{6}$$

Isso assegura pressão seletiva moderada e preserva a diversidade genética.

2. Crossover Híbrido ($P_{\text{cross}} = 0{,}90$):
Sorteia-se $r \sim \mathcal{U}(0, 1)$. Se $r > P_{\text{cross}}$, os descendentes são cópias idênticas dos pais. Se $r \leq P_{\text{cross}}$, aplicam-se operadores especializados em cada metade do cromossomo:

- **Parte Binária (Cruzamento Uniforme):** Para cada gene $j \in \{1, \ldots, D\}$, troca-se os alelos entre os pais com probabilidade 0,50:

$$(f_{1,j}, f_{2,j}) = \begin{cases} (p_{2,j}, p_{1,j}), & \text{se } u_j < 0{,}50 \\ (p_{1,j}, p_{2,j}), & \text{caso contrário} \end{cases}, \quad u_j \sim \mathcal{U}(0, 1) \tag{7}$$

- **Parte Contínua (Blend Crossover – BLX-α, $\alpha = 0{,}50$):** Para cada hiperparâmetro $k \in \{D + 1, \ldots, D + K\}$:

$$c_{\min} = \min(p_{1,k}, p_{2,k}), \quad c_{\max} = \max(p_{1,k}, p_{2,k}) \tag{8}$$

$$I = c_{\max} - c_{\min} \tag{9}$$

$$f_{1,k} = \text{clip}\left(\mathcal{U}(c_{\min} - \alpha I,\ c_{\max} + \alpha I),\ 0{,}0,\ 1{,}0\right) \tag{10}$$

$$f_{2,k} = \text{clip}\left(\mathcal{U}(c_{\min} - \alpha I,\ c_{\max} + \alpha I),\ 0{,}0,\ 1{,}0\right) \tag{11}$$

Propriedade Matemática: O BLX-α com $\alpha = 0{,}50$ amostra em um intervalo de amplitude $2I$, combinando a **explotação** da região interna entre os pais com a **exploração** das bordas externas, prevenindo a contração do hipercubo.

3. Operadores de Mutação Híbridos:

- **Parte Binária (Mutação Bit-Flip):** Cada posição $j \in \{1, \ldots, D\}$ inverte seu estado lógico com taxa $P_{\text{mut}} = 0{,}10$:

$$m_j \leftarrow \begin{cases} 1 - m_j, & \text{se } r_j < 0{,}10 \\ m_j, & \text{caso contrário} \end{cases}, \quad r_j \sim \mathcal{U}(0,1) \tag{12}$$

- **Parte Contínua (Perturbação Gaussiana Truncada):** Para cada gene contínuo $k \in \{D+1, \ldots, D+K\}$, adiciona-se ruído normal com desvio padrão $\sigma = 0{,}10$:

$$\gamma_k \leftarrow \text{clip}\left(\gamma_k + \mathcal{N}(0, \sigma^2),\ 0{,}0,\ 1{,}0\right), \quad \sigma = 0{,}10 \tag{13}$$

4. Mecanismo de Elitismo ($N_{\text{elite}} = 2$):
Os 2 indivíduos de maior aptidão da geração atual g são clonados sem modificações para a geração $g+1$, garantindo que a aptidão máxima seja monotonicamente não-decrescente:

$$\mathcal{F}_{\max}(g+1) \geq \mathcal{F}_{\max}(g), \quad \forall g \tag{14}$$

1.5 Função de Aptidão (Fitness) e Validação Cruzada Estratificada

A função objetivo a ser maximizada é o F_1-macro médio obtido via validação cruzada estratificada em 5 dobras ($K_{\text{cv}} = 5$) avaliada **exclusivamente sobre o conjunto de treino**:

$$\mathcal{F}(\mathbf{c}) = \frac{1}{K_{\text{cv}}} \sum_{k=1}^{K_{\text{cv}}} F_{1,\text{macro}}^{(k)}(\mathbf{X}_{\text{train}}[:, \mathbf{m}],\ \mathbf{y}_{\text{train}};\ \boldsymbol{\theta}(\boldsymbol{\gamma})) \tag{15}$$

com a métrica multiclasse:

$$F_{1,\text{macro}} = \frac{1}{C} \sum_{c=1}^{C} \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \tag{16}$$

Se o cromossomo violar a restrição de viabilidade física ($\sum_{j=1}^{D} m_j < 2$), sua aptidão é penalizada para $\mathcal{F}(\mathbf{c}) = 0{,}0$.

2 Pilar 2: Rastreamento no Código (Linha a Linha)

2.1 Mapeamento de Arquivos e Estruturas

- **`src/iga_xgboost/cromossomo.py`:** Especificação do espaço de busca (`HPARAM_SPECS`), geração enviesada (`gerar_cromossomo`), decodificação afim (`decodificar`) e teste de viabilidade (`validar`).
- **`src/iga_xgboost/iga.py`:** Classe principal `IGA`, loop de gerações (`evoluir`), operadores (`_selecionar`, `_cruzar`, `_mutar`) e cálculo de aptidão via pipeline (`_avaliar`).
- **`src/iga_xgboost/train_classification.py`:** Pipeline do Track B acoplando `CustomSMOTE` com o IGA.

2.2 Rastreamento da Decodificação Afim (`src/iga_xgboost/cromossomo.py`)

O trecho abaixo transforma o cromossomo contínuo em hiperparâmetros utilizáveis pelo XGBoost:

```python
def decodificar(cromossomo: np.ndarray, track: str) -> tuple[list[str], dict]:
    features, _ = _specs_do_track(track)
    n_features = len(features)

    # Separacao do cromossomo: parte binaria e parte continua
    bits = cromossomo[:n_features]
    continuo = cromossomo[n_features:]

    # Variaveis ativas: bits >= 0.5 sao consideradas selecionadas
    features_ativas = [f for f, b in zip(features, bits) if b >= 0.5]

    # Mapeamento afim dos hiperparametros
    hparams: dict = {}
    for i, (nome, lo, hi, tipo) in enumerate(HPARAM_SPECS):
        val = lo + float(continuo[i]) * (hi - lo)
        hparams[nome] = int(round(val)) if tipo == "int" else float(val)

    return features_ativas, hparams
```

2.3 Rastreamento dos Operadores Genéticos (`src/iga_xgboost/iga.py`)

Os operadores de cruzamento e mutação implementam a bipartição do cromossomo:

```python
def _cruzar(self, pai1: np.ndarray, pai2: np.ndarray) -> tuple[np.ndarray,
    np.ndarray]:
    if self.rng.random() > P_CROSSOVER:  # P_CROSSOVER = 0.90
        return pai1.copy(), pai2.copy()

    f1, f2 = pai1.copy(), pai2.copy()

    # 1. Parte Binaria: Cruzamento Uniforme (troca de bits com p=0.5)
    for i in range(self.n_features):
        if self.rng.random() < 0.5:
            f1[i], f2[i] = f2[i], f1[i]

    # 2. Parte Continua: Crossover BLX-alpha (alpha = 0.50)
    for i in range(self.n_features, self.n_cromossomo):
        lo = min(pai1[i], pai2[i])
        hi = max(pai1[i], pai2[i])
        ext = BLX_ALPHA * (hi - lo)
        f1[i] = np.clip(self.rng.uniform(lo - ext, hi + ext), 0.0, 1.0)
        f2[i] = np.clip(self.rng.uniform(lo - ext, hi + ext), 0.0, 1.0)

    return f1, f2

def _mutar(self, cromossomo: np.ndarray) -> np.ndarray:
    c = cromossomo.copy()
    # 1. Mutacao Bit-Flip na parte binaria (p = 0.10 por bit)
    for i in range(self.n_features):
        if self.rng.random() < P_MUTACAO_BIT:
            c[i] = 1.0 - c[i]

    # 2. Perturbacao Gaussiana na parte continua (sigma = 0.10)
    for i in range(self.n_features, self.n_cromossomo):
        c[i] = np.clip(c[i] + self.rng.normal(0.0, SIGMA_MUTACAO), 0.0, 1.0)

    return c
```

2.4 Rastreamento do Pipeline de Avaliação de Fitness (`src/iga_xgboost/iga.py`)

A função `_avaliar()` assegura que a seleção de atributos ocorra estritamente dentro de cada dobra de validação cruzada:

```python
def _avaliar(self, cromossomo: np.ndarray) -> float:
    if not validar(cromossomo, self.track):
        return 0.0

    features_ativas, hparams = decodificar(cromossomo, self.track)
    idx_features = [self.features.index(f) for f in features_ativas]

    # Seletor posicionado DENTRO do pipeline de CV
    selector = FunctionTransformer(lambda X, idx=idx_features: X[:, idx])

    clf = XGBClassifier(
        **hparams,
        tree_method="hist",
        eval_metric="mlogloss",
        random_state=self.seed,
        n_jobs=1,
        verbosity=0,
    )

    if self.usar_smote:
        sampler = self.smote_sampler if self.smote_sampler is not None \
                  else SMOTE(random_state=self.seed,
                             k_neighbors=self.smote_k_neighbors)
        # O CustomSMOTE e o Seletor operam por fold
        pipeline = ImbPipeline([
            ("smote", sampler),
            ("select", selector),
            ("clf", clf),
        ])
    else:
        pipeline = ImbPipeline([("select", selector), ("clf", clf)])

    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=self.seed)
    scores = cross_val_score(
        pipeline, self.X_train, self.y_train,
        cv=cv, scoring="f1_macro", n_jobs=-1
    )
    return float(np.clip(scores.mean(), 0.0, 1.0))
```

2.5 Tabela de Dimensões e Configurações por Eixo

A Tabela 1 sintetiza os parâmetros estruturais do IGA nos diferentes cenários avaliados no TCC.

Tabela 1: Parâmetros Estruturais e Dimensões do IGA nos Cenários do TCC.

Cenário	Genes D	Genes K	Total	Pop.	Variáveis Selecionadas (p^*)
Track A: Completo (C1)	17	9	26	50	8 features (CO, 3 razões, 4 LSTM)
Track A: Sem LSTM (C5b)	9	9	18	50	8 features (4 gases brutos + 4 razões)
Track B: Completo (C3)	19	9	28	50	16 features (5 gases + 4 razões + 7 Duval)
Track B: Baseline Wu (M3)	18	9	27	50	14 features (selecionadas de 18 candidatas)

3 Pilar 3: Decisões de Projeto e Nuances do Meu Trabalho

3.1 Por que IGA e não Otimização Bayesiana (Optuna) ou GridSearchCV?

Justificativa da Escolha do Algoritmo Genético Aprimorado

1. **Busca Simultânea Mista (Binária + Contínua):** Algoritmos de busca em grade (GridSearchCV) tornam-se inviáveis quando o espaço combinatório envolve $2^{19} = 524.288$ subconjuntos de atributos multiplicados por combinações de hiperparâmetros.

2. **Limitação da Otimização Bayesiana Pura (TPE / GP):** Processos Gaussianos e TPE (como implementados no Optuna) são altamente eficientes para espaços contínuos euclidianos suaves. Contudo, quando acoplados a máscaras binárias esparsas de alta dimensionalidade ($\{0,1\}^{19}$), as funções de covariância (kernels) sofrem degradação por descontinuidade, convergindo para mínimos locais onde apenas 1 ou 2 variáveis são ativadas.

3. **Capacidade Exploratória e Preservação com IGA:** O algoritmo genético opera com uma população de 50 indivíduos distribuídos em múltiplos vales do espaço de busca. O operador BLX-α preserva a exploração contínua sem contrair a variância populacional, e o elitismo ($N_{\text{elite}} = 2$) garante que boas combinações de atributos não sejam destruídas.

3.2 O Papel Físico da Inicialização Enviesada ($p_{\text{iec}} = 0{,}80$)

Por que favorecer as razões normativas na geração inicial?

Se a população inicial sorteasse bits com $p = 0{,}50$ em todas as posições, a probabilidade de um indivíduo nascer sem nenhuma das razões padronizadas da norma IEC 60599 seria significativa. Na prática de transformadores de potência, décadas de engenharia de alta tensão comprovam que as relações estequiométricas (como C_2H_2/C_2H_4) são indicadores de temperatura muito mais estáveis do que as concentrações absolutas isoladas, que variam com o volume do tanque. Ao configurar $p_{\text{iec}} = 0{,}80$ e $p_{\text{demais}} = 0{,}50$, o algoritmo explora preferencialmente o espaço de soluções fisicamente defensáveis, acelerando a busca sem restringir o espaço ótimo.

3.3 A Relação entre Complexidade Computacional e Ganho Diagnóstico

O IGA justificou o custo computacional no Track B?

Os Fatos Numéricos (Tabela 7 e 8 do TCC):

- XGBoost Default (C4, parâmetros fixos da literatura, 19 variáveis): $F_{1,\text{macro}} = 0{,}8855$.
- XGBoost Otimizado por IGA (C3, 16 variáveis selecionadas): $F_{1,\text{macro}} = 0{,}8909$.
- Ganho Absoluto de Desempenho: $\Delta F_1 = +0{,}0054$ (diferença estatisticamente nula via bootstrap, $p > 0{,}05$).

Onde Residuo o Real Mérito de Engenharia:
O valor do IGA não esteve em inflar a métrica em 10 pontos percentuais, mas sim em:

1. **Descarte de 3 Variáveis Irrelevantes:** O IGA eliminou C_2H_2, C_2H_2/CH_4 e CH_4/TH_1, simplificando o classificador sem perda preditiva.
2. **Redução Drástica da Sensibilidade a Hiperparâmetros:** O IGA identificou que configurações com alta profundidade ($d_{\max} = 12$) associadas a subamostragens agressivas de colunas ($s_{\text{col}} = 0{,}34$) e forte regularização L_2 ($\lambda = 3{,}94$) estabilizam o classificador contra a sobreposição das classes térmicas T1 e T2.

3.4 Alinhamento Rigoroso entre Código e Texto do TCC

- **Critério de Parada e Gerações:** O código (`src/config.py`) define `IGA_MAX_GERACOES = 100` e `IGA_GERACOES_SEM_MELHORIA = 20`, com população `IGA_POPULACAO = 50`. No texto do TCC (`metodologia.tex`, linha 127), a descrição reflete exatamente esses valores: *"população de 50 indivíduos por até 100 gerações, incluindo critério de parada se a melhor aptidão não apresentar melhoria após 20 gerações consecutivas"*.
- **Posicionamento do CustomSMOTE:** No Track B, a chamada de superamostragem sintética foi inserida dentro do `ImbPipeline` no método `_avaliar()` do IGA, garantindo que o treinamento dos 5 folds respeite o isolamento das amostras de validação interna.

4 Pilar 4: Sabatina da Banca (3 Perguntas Difíceis e Gabaritos)

Pergunta Crítica da Banca: 1. Crossover BLX-α: Matemática de Amostragem e Fronteiras

"O seu código utiliza o operador Blend Crossover (BLX-α) com $\alpha = 0{,}50$ para recombinar os 9 genes de hiperparâmetros contínuos. Explique analiticamente como esse operador expande o intervalo dos pais. Se o espaço de busca dos hiperparâmetros está normalizado no hipercubo unitário $[0,1]^9$, o que impede que o fator de expansão $\alpha = 0{,}50$ ou a mutação gaussiana gerem valores fora do domínio válido (como $\gamma_k < 0$ ou $\gamma_k > 1$)? Como o seu código lida com os hiperparâmetros inteiros?"

Gabarito Técnico Ideal de Resposta (Blindagem de Banca)

Estratégia de Resposta:
Apresente a dedução analítica do intervalo estendido do BLX-α, evidencie a operação de projeção não-linear (*clipping*) e detalhe o mapeamento discretizado:

1. **Mecanismo de Expansão do BLX-α:** "Sejam dois progenitores com alelos contínuos $p_{1,k}$ e $p_{2,k}$. Definindo $c_{\min} = \min(p_{1,k}, p_{2,k})$, $c_{\max} = \max(p_{1,k}, p_{2,k})$ e a distância $I = c_{\max} - c_{\min}$, o operador BLX-α realiza um sorteio uniforme contínuo no intervalo

expandido:

$$f_k \sim \mathcal{U}\left(c_{\min} - \alpha I,\ c_{\max} + \alpha I\right) \tag{17}$$

Com $\alpha = 0{,}50$, a amplitude total da distribuição é:

$$(c_{\max} + 0{,}5I) - (c_{\min} - 0{,}5I) = I + 1{,}0I = 2I \tag{18}$$

Isso significa que 50% da probabilidade recai na região interna de interpolação entre os pais (*explotação*) e 50% recai nas regiões externas além dos pais (*exploração*), garantindo que a população continue explorando o espaço sem contração prematura da variância."

2. **Tratamento de Violação de Fronteiras no Código:** "Como $c_{\min} - 0{,}5I$ pode resultar em valor negativo e $c_{\max} + 0{,}5I$ pode exceder 1,0, o código aplica a função de projeção não-linear `np.clip(val, 0.0, 1.0)` imediatamente após a amostragem (linha 248 de `src/iga_xgboost/iga.py`). A mesma operação de saturação é aplicada após a perturbação da mutação gaussiana:

$$\gamma_k = \min\left(1{,}0,\ \max\left(0{,}0,\ \gamma_k + \mathcal{N}(0, \sigma^2)\right)\right) \tag{19}$$

garantindo estritamente $\gamma_k \in [0{,}0; 1{,}0]$ antes de qualquer etapa subsequente."

3. **Mapeamento dos Hiperparâmetros Inteiros:** "Na função `decodificar()` em `cromossomo.py`, os genes contínuos normalizados correspondentes a parâmetros discretos (como `max_depth`, `n_estimators` e `min_child_weight`) passam pela transformação afim $lo + \gamma_k \cdot (hi - lo)$ e são arredondados para o inteiro mais próximo através de `int(round(val))`, mapeando o segmento contínuo para o reticulado discreto de forma homogênea."

Pergunta Crítica da Banca: 2. O Custo Computacional do IGA vs. O Empate Técnico com o Baseline

"Avaliando os resultados do Track B na Tabela 7 do seu TCC, o classificador XGBoost com configuração de referência fixa (C4, sem otimização evolutiva) atinge um F_1-macro de 0,8855, enquanto o modelo otimizado por IGA (C3) atinge 0,8909. O teste pareado via bootstrap demonstra que esse ganho de apenas +0,0054 não possui significância estatística ($\Delta F_1 \in [-0{,}015; +0{,}026]$). Se o IGA executa 50 indivíduos por até 100 gerações com 5 dobras de validação cruzada — totalizando até 25.000 treinamentos completos de XGBoost —, como você justifica o enorme custo computacional desse algoritmo perante a banca se o resultado final foi praticamente um empate técnico com os hiperparâmetros padrão?"

Gabarito Técnico Ideal de Resposta (Blindagem de Banca)

Estratégia de Resposta:
Reconheça a precisão dos números do examinador e redirecione a defesa para a dimensionalidade, estabilidade operacional e eliminação de custos físicos de sensores:

1. **O Objetivo Primário do IGA Não Era Apenas Aumentar Métrica:** "A observação da banca é perfeitamente exata quanto ao valor do F_1-macro no conjunto de teste. Contudo, o objetivo do Algoritmo Genético Aprimorado em Engenharia não é apenas maximizar uma métrica isolada, mas sim encontrar a **fronteira de Pareto entre desempenho preditivo e complexidade dimensional**."

2. **Redução de Dimensionalidade e Custo Prático de Manutenção:** "O modelo C4 de referência operou sobre todas as 19 variáveis candidatas. Já o IGA reduziu o vetor

de entrada para 16 variáveis no Track B e para 8 variáveis no Track A (uma redução de mais de 50% dos atributos), mantendo o mesmo patamar de desempenho de $F_1 \approx 0{,}89$. Na prática de subestações de energia, descartar variáveis significa que a concessionária não precisa instalar analisadores cromatográficos de alto custo para monitorar gases que não agregam informação discriminativa."

3. **Descoberta da Região de Regularização Ótima:** "O XGBoost default utiliza regularização L_2 unitária ($\lambda = 1{,}0$) e profundidade fixa ($d = 6$). O IGA descobriu que, na presença de classes raras e sobrepostas como T1 e T2, a melhor arquitetura exige árvores mais profundas ($d = 12$), porém compensadas por forte regularização Ridge ($\lambda = 3{,}94$) e amostragem restrita de colunas ($s_{\text{col}} = 0{,}34$). Essa combinação previne o sobreajuste e estabiliza o modelo contra o ruído de medição cromatográfica, algo que uma parametrização arbitrária manual não conseguiria garantir."

Pergunta Crítica da Banca: 3. Vazamento de Seleção na Validação Cruzada Repetida (5×5)

"Na Seção 3.6.3 do TCC, você afirma que o IGA selecionou as variáveis e os hiperparâmetros usando validação cruzada 5-fold sobre o conjunto de treino. Porém, na Seção 4.3, você apresenta uma validação cruzada repetida 5×5 (25 dobras) para diagnosticar a estabilidade do modelo, mantendo as features e hiperparâmetros fixados previamente. Isso não introduz viés de seleção (selection bias)? Por que você não re-executou a seleção do IGA dentro de cada repetição externa da validação cruzada?"

Gabarito Técnico Ideal de Resposta (Blindagem de Banca)

Estratégia de Resposta:
Demonstre total domínio sobre a teoria de Validação Cruzada Aninhada (*Nested Cross-Validation*), esclarecendo a função diagnóstica da CV repetida e garantindo a inviolabilidade do conjunto de teste independente:

1. **O Teste Independente Permaneceu Estritamente Cego:** "O ponto metodológico mais crítico do nosso trabalho é que o conjunto de teste oficial (as 295 amostras deduplicadas do Track B e as 900 séries do Track A) jamais foi acessado pelo IGA. Todo o processo de seleção de atributos e afinação de hiperparâmetros ocorreu estritamente sobre a partição de treino/desenvolvimento. Portanto, o teste final independente é 100% não enviesado e reflete uma generalização autêntica."

2. **Objetivo da Validação Cruzada Repetida 5×5:** "A validação cruzada repetida apresentada na Seção 4.3 não teve a finalidade de estimar o erro de generalização de um pipeline de busca, mas sim de atuar como uma **ferramenta de diagnóstico de estabilidade estocástica** do classificador sob diferentes partições dos dados de treino. Nós documentamos com total transparência no texto que o subconjunto de variáveis foi mantido fixo para isolar a variabilidade da amostragem em relação à variabilidade da busca heurística."

3. **Inviabilidade de uma Validação Cruzada Aninhada Integral:** "Uma validação aninhada estrita exigiria rodar o IGA dentro de cada uma das 25 dobras externas. Como uma única execução do IGA já executa até 100 gerações $\times$ 50 indivíduos $\times$ 5 folds = 25.000 treinamentos de XGBoost, aninhar esse procedimento exigiria $25 \times 25.000 =$ **625.000** treinamentos de modelos XGBoost acoplados ao CustomSMOTE. Esse custo computacional seria desproporcional para demonstrar o que o teste independente de 295

amostras já comprovou com rigor: o modelo atinge o mesmo patamar de desempenho dos métodos normativos operando com um subconjunto reduzido de variáveis."

Fim do Módulo 3 – Domínio Completo do Algoritmo Genético Aprimorado!